

复旦大学计算机科学技术学院

学年第 学期期末试卷

课程名称：_____ 课程代码：_____

卷 别： ☐A 卷 ☐B 卷 ☒C 卷 ☐D 卷

姓 名：_____ 学 号：_____

提示：请同学们秉持诚实守信宗旨，谨守考试纪律，摒弃考试作弊。学生如有违反学校考试纪律的行为，学校将按《复旦大学学生纪律处分条例》规定予以严肃处理。

题号	一			二		三	四			总分
	1	2	3	1	2	1	1	2	3	
得分										

一、阅读程序写输出（30%）

第1题（10%）

#假设用户输入为：0（Enter） 3（Enter）8（Enter）5（Enter）。

```
secret = 5
```

```
guess_times = 0
```

```
while guess_times < 3:
    num = int(input("猜一个数字："))
    if num < 1 or num > 10:
        print("输入无效，跳过此轮。")
        continue
    if num == secret:
        print("猜对了！")
        break
    else:
        print("猜错了。")
    guess_times += 1
```

答案:

输入无效, 跳过此轮。

猜错了。

猜错了。

猜对了!

第 2 题 (10%)

阅读下面的程序代码, 写出程序运行后的输出结果。

```
def slice_puzzle(data):
    temp1 = data[3:15:2]
    temp2 = temp1[::-1]
    result1 = temp2[1:4]

    indices = [i for i in range(len(data)) if data[i] % 3 == 0]
    selected = [data[i] for i in indices]
    result2 = selected[2:-2:2]

    first_half = data[:len(data)//2]
    second_half = data[len(data)//2:]
    mixed = first_half[::-2] + second_half[::2]
    result3 = mixed[2:7]

    return result1, result2, result3

numbers = [3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48]
r1, r2, r3 = slice_puzzle(numbers)
print("操作 1 的结果:", r1)
print("操作 2 的结果:", r2)
print("操作 3 的结果:", r3)
```

参考答案

操作 1 的结果: [36, 30, 24]

操作 2 的结果: [9, 15, 21, 27, 33, 39]

操作 3 的结果: [15, 21, 48, 42, 36]

第 3 题 (10%)

```
data = {"m": 6, "d": 18., "y": 2025., "hi": 5.5, "entity": 11}
count, step, results = 0, 10, []
# 假定遍历字典的顺序为: m, d, y, hi, entity
for key, value in data.items():
```

```

    if (len(key) == 1):
        if value>10:
            results.append(value - step)
        else:
            results.append(value + step)
    else:
        if isinstance(value, str):
            results.append(len(value))
            continue
        else:
            results.append(value + step)

print("输出结果:", results)
print("结果总和:", sum([v for v in results if isinstance(v, (int, float))]))

```

参考答案:
输出结果: [16, 8.0, 2015.0, 15.5, 21]
结果总和: 2075.5
 ...

说明:

从 python3.7 开始, 枚举字典中的(key, value)时, 其顺序与插入的顺序一致。实际上, 最终的和与不依赖与顺序, 输出结果(列表)的数值不依赖顺序。

考点:

数据处理; 循环; 分支; 列表推导式; 内置函数; 类型;
 ...

二、程序填空 (10%)

```
# -*- coding: utf-8 -*-
```

```
'''-----
```

下述代码片段实现如下功能:

(1) 调用函数 `get_response(name)` 获得响应字符串 `response`。下面给出了响应的格式示例:

```
name=tony/1/2/3/31.456
```

```
name=tom/1/45
```

```
name=alice/1/2/0
```

```
name=charlie/75.6
```

```
name=unknown
```

(2) 分析响应, 从中获取 `stat` (`stat` 为响应中最后一个/之后的数值)。

(3) 将 `name` 和 `stat` 按照指定的格式输出: `name` 居中对齐, 占 10 个字符的宽度; `stat` 小数点后保留 2 位, 右对齐, 占 10 个字符的宽度。

请基于上面的描述补全缺失的部分。对于上述响应示例，该程序运行结果如下所示：

```
tony      31.46
tom       45.00
alice     0.00
charlie   75.60
```

注意：

- (1) 每行限填一行代码。
- (2) 除要求填空的位置之外，请勿改动程序中的其他内容。
- (3) 请用字符串的 **format** 方法实现字符串格式化

```
names = ['tony', 'tom', 'alice', 'charlie']
for name in names:
    response = get_response(name)
    _____
    if idx != -1:
        stat = float(response[idx+1:])
        _____
```

参考答案

- 1. `idx = response.rfind('/')`
- 2. `print('{:^10}{:>10.2f}'.format(name, stat))`

三、程序改错（20%）

第1题（10%）

```
"""
```

以下程序的目的是找出正整数 **M** 和 **N** 之间（含 **N** 和 **M**，且 **N** 不小于 **M**）的所有真素数。

真素数的定义：如果一个正整数 **P** 为素数，且其反序也为素数，那么 **P** 就为真素数。
例如，**11**，**13** 均为真素数，因为 **11** 的反序还是为 **11**，**13** 的反序为 **31** 也为素数。

以下程序有两处错误，请在答题位置填写错误行的编号（每行最右边注释里的数字。如果代码行后面没有注释，表示该行不会出错）和改正后的整行代码。改正后的代码只能是一行，不允许跨越多行。

```
"""
def is_prime(n):
    """判断一个数是否为素数"""
```

```

if n < 2:
    return False
for i in range(2, int(n ** 0.5) + 1): # 1
    if n % i == 0: # 2
        return False # 3
return True

def reverse_number(n):
    """返回一个数的反序数"""
    return int(str(n)[::-1])

def find_true_primes(left, right):
    """找出 M 到 N 之间的所有真素数"""
    true_primes = []
    for num in range(left, right): # 4
        if is_prime(num) and is_prime(reverse_number(num)): # 5
            true_primes.append(num) # 6
    return true_primes

if __name__ == '__main__':

    M, N = 3, 31
    result = find_true_primes(left = M, N) # 7

    if result:
        print("在{}到{}之间的真素数有：".format(M, N))
        print(*result)
    else:
        print("在{}到{}之间没有找到真素数".format(M, N))

"""
输出结果应该为：
在 3 到 31 之间的真素数有：
3 5 7 11 13 17 31
"""

```

错误行 编号	改正
4	for num in range(left, right + 1)
7	result = find_true_primes(M, N) 或者 result = find_true_primes(left = M, right = N)

第 2 题 (10%)

'''-----

【程序改错】

以下程序通过列表推导从嵌套列表中提取所有评分大于 8 的电影名称（转为大写），并检查指定电影 **target** 是否存在（不区分大小写）。

以下程序有两处错误，请在答题位置填写错误行的编号(每行最右边注释里的数字。如果代码行后面没有注释，表示该行不会出错)和改正后的整行代码。改正后的代码只能是一行，不允许跨越多行。

-----'''

```
# 原始数据
movies = [{"Ne Zha 2", 9.2}, {"interstellar", 8.6}, {"dunkirk", 7.8}]
target = "INTERSTELLAR"

# 提取评分>8 的电影名称（大写）
highRated=[title.upper() for title in film in movies if film[1]>8] #1
print("高评分电影：{0}".format(highRated)) #2

# 检查是否存在目标电影（大小写不敏感）
if target.lower() in highRated: #3
    print("电影 '{}' 在榜单中".format(target)) #4
else: #5
    print("电影 '{}' 不在榜单中".format(target)) #6
'''-----
```

输出结果应该为：
高评分电影： ['NE ZHA 2', 'INTERSTELLAR']
电影 'INTERSTELLAR' 在榜单中

-----'''

错误行 编号	改正
1	highRated=[film[0].upper() for film in movies if film[1]>8] 或 highRated=[title.upper() for title, score in movies if score>8]
3	if target.upper() in highRated:

四、编程 (40%)

第 1 题 (10%)

BMI 计算

第 1 步取得输入：身高和体重数据

说明：身高和体重在同一行输入，用空格隔开，身高单位为米，体重单位为公斤。假设输入的数据都是正确的，即不需要处理输入数据错误的情况。

第 2 步计算：BMI=体重/身高平方。

第 3 步根据算出的 BMI 做健康提醒

说明：BMI 的正常范围为[18.5-23.9]。

如果大于 BMI 的正常范围，则输出提醒“BMI 指数为...，太重啦！少吃、多运动！”；

如果小于 BMI 的正常范围，则输出提醒“BMI 指数为...，太轻啦！多吃、多睡、多运动！”；

”；

如果体重正常，则输出提醒“BMI 指数为...，正常！保持住！”。

程序运行示例如下（身高和体重数据是用户输入，其它为程序输出的提示和信息）：

运行示例 1：

请输入身高（米）和体重（公斤），用空格分隔：1.7 100

BMI 指数为 34.6，太重啦！少吃、多运动！

运行示例 2：

请输入身高（米）和体重（公斤），用空格分隔：1.7 60

BMI 指数为 20.8，正常！保持住！

运行示例 3：

请输入身高（米）和体重（公斤），用空格分隔：1.7 40

BMI 指数为 13.8，太轻啦！多吃、多睡、多运动！

参考答案：

获取用户输入的身高和体重

```
height, weight = input("请输入身高（米）和体重（公斤），用空格分隔：").split()
```

```
height = float(height)
```

```
weight = float(weight)
```

计算 BMI 指数

```
bmi = weight / (height ** 2)
```

根据 BMI 指数给出健康建议

```
if bmi > 23.9:
```

```
    print(f"BMI 指数为{bmi:.1f}，太重啦！少吃、多运动！")
```

```
elif bmi < 18.5:
```

```
    print(f"BMI 指数为{bmi:.1f}，太轻啦！多吃、多睡、多运动！")
```

```
else:
```

```
    print(f"BMI 指数为{bmi:.1f}，正常！保持住！")
```

第 2 题（10%）

合并多个用户的购物车，统计每种商品的总数量。

请编写一个函数，将多个购物车合并成一个购物车。
仅需编写在 Program, End 注释标志之间的若干语句。

```
def merge_carts(*carts):  
    ***** Program *****
```

```
***** End *****
```

测试代码如下：

测试

```
cart1 = {"苹果": 3, "香蕉": 2, "橘子": 1}
```

```
cart2 = {"香蕉": 1, "葡萄": 4}
```

```
cart3 = {"火龙果": 2}
```

```
print(merge_carts(cart1, cart2, cart3))
```

```
# 输出: {'苹果': 3, '香蕉': 3, '橘子': 1, '葡萄': 4, '火龙果': 2}
```

===== 参考答案 =====

【评分标准】每行代码 2 分。

```
def merge_carts(*carts):  
    merged = {}  
    for cart in carts:  
        for item, quantity in cart.items():  
            merged[item] = merged.get(item, 0) + quantity # 使用 get()  
    累加商品数量  
    return merged
```

C 卷第 3 题 (20%)

```
# -*- coding: utf-8 -*-
```

```
'''-----
```

【程序设计】

1. 题目要求：为了准备校庆活动，校友会正在整理历届毕业校友的登记信息，所有信息保存在一个文本文件 `alumni.txt` 中。请编写程序按年份统计校友人数和输出校友列表。请按照以下要求完善相应的函数，实现对应的功能。

2. 文件格式说明，每行表示一位校友的登记记录，格式如下：[毕业年份]姓名(毕业院系)<手机号>。例如："[2010]张三(计算机学院)<13812345678>"。其中，

- 毕业年份：4 位数字
- 毕业院系：英文圆括号中的字符串

- 手机号：必须是合法的大陆手机号，约定为以 13/15/18/19 开头的 11 位数字。

3. 具体编程要求如下：

3.1 请完善 `parse_line` 函数，要求使用正则表达式解析并验证一行校友信息。函数返回：元组(`year`, `name`, `dept`, `phone`) 或 `None`。当判断格式不合法时，返回 `None`。

3.2 请完善 `read_alumni_file` 函数，读取文件，统计校友信息。返回一个按年份分类的字典，格式如下：{`year`: [(`name`, `dept`, `phone`), ...]}。要求：（1）先判断是否重复登记，如果重复，则跳过并输出重复记录提示；（2）再调用 `parse_line` 函数判断是否合法。对非法格式或非法手机号，跳过并输出格式错误提示；（3）要求能处理文件不存在的异常。

3.3 请完善 `print_sorted_alumni` 函数，按年份排序输出统计信息，同年的校友列表按院系排序。

4. 示例

==》若文件 `alumni.txt` 中内容如下==》

```
[2010]张三(计算机学院)<13812345678>
[2012]李四(数学系)<15898765432>
[2010]王五(计算机学院)<13900000000>
[2010]王海(物理)<13900000168>
[2012]张一(化学)<15812345678>
[2010]张三(计算机学院)<13812345678>
[2012]杨武(化学)<18812348765>
[1988]李宁(化学)<15912345>
[1998]李明(生物)<19812345678>
[1998]李明(生物)<14812345678>
```

==》最终输出如下==》

```
重复记录，已跳过行： [2010]张三(计算机学院)<13812345678>
格式错误，已跳过行： [1988]李宁(化学)<15912345>
格式错误，已跳过行： [1998]李明(生物)<14812345678>
```

==校友列表如下==

备注：按年份排序输出人数统计，校友按院系排序。

1998 年毕业人数： 1

- 李明(生物):19812345678

2010 年毕业人数： 3

- 王海(物理):13900000168
- 张三(计算机学院):13812345678
- 王五(计算机学院):13900000000

2012 年毕业人数: 3

- 张一(化学):15812345678
- 杨武(化学):18812348765
- 李四(数学系):15898765432

注意: 部分源程序给出如下。请勿改动主函数 `main` 和其它函数中的
任何内容, 仅在函数的注释标志之间填入所编写的若干语句。

```
import re

def parse_line(line):
    """
    解析并验证一行校友信息。
    返回: (year, name, dept, phone) 或 None
    """
    *****Program*****

    *****End*****

def read_alumni_file(filename):
    """
    读取文件, 统计校友信息。
    返回一个按年份分类的字典: {year: [(name, dept, phone), ...]}
    """
    alumni_dict = {}

    *****Program*****

    *****End*****

    return alumni_dict

def print_sorted_alumni(alumni_dict):
    """
    按年份排序输出统计信息, 校友按院系排序。
    """
```

```
print("==校友列表如下==\n 备注：按年份排序输出人数统计，校友按院系排序。")
```

```
*****Program*****
```

```
*****End*****
```

```
# 主程序入口
```

```
if __name__ == "__main__":  
    filename = "alumni.txt"  
    alumni = read_alumni_file(filename)  
    print()  
    print_sorted_alumni(alumni)
```

答案 1 (parse_line) :

```
pattern =
re.compile("\[(\d{4})\](.+)\((.+)\)<(1[3|5|8|9]\d{9})>")
match = pattern.match(line.strip())
if match:
    return match.groups() # (year, name, dept, phone)
else:
    return None
```

或者

```
pattern =
re.compile("\[(\d{4})\](.+)\((.+)\)<(1[3|5|8|9]\d{9})>")
match = pattern.match(line.strip())#strip 可以不用

if match:
    year = match.group(1)
    name = match.group(2)
    dept = match.group(3)
    phone = match.group(4)
    return (year, name, dept, phone)
else:
    return None
```

答案 2 (read_alumni_file) :

seen = set() # 用于去重

```
try:
    with open(filename, 'r') as f:
        for line in f: #f 可以换成 f.readlines()
            if line in seen:
                print("重复记录, 已跳过行: ", line.strip())
                continue
            seen.add(line)

            data = parse_line(line)
            if not data:
                print("格式错误, 已跳过行: ", line.strip())
                continue
            year, name, dept, phone = data
```

```

        if year not in alumni_dict:
            alumni_dict[year] = []          #整个 if 语句, 可以
替换为 alumni_dict[year] = alumni_dict.get(year, [])
            alumni_dict[year].append((name, dept, phone))
except OSError:    #可以换成 FileNotFoundError
    print("文件未找到: ", filename)
    return {}

```

答案 3 (print_sorted_alumni):

```

for year in sorted(alumni_dict.keys()):
    alumni_list = alumni_dict[year]
    # 按院系排序(sorted 函数可以换成 sort 函数, 但形式有变化)
    alumni_list_sorted = sorted(alumni_list, key=lambda
x: x[1])
    print(year, "年毕业人数: ", len(alumni_list_sorted))
    for name, dept, phone in alumni_list_sorted:
        print("    - %s(%s):%s" % (name, dept, phone))    #
可以使用 format 函数

```